

Project : Extended NBA Player Performance Analysis Using Python

Dataset: NBA Players Dataset (`all_seasons.csv`)

Background and Problem Definition

For this Project, I extended my Project 'An Analysis of Factors Influencing NBA Player Performance' topic by analyzing NBA player performance using Python instead of R. The goal of this project is to use **pandas** for data manipulation, **matplotlib** for visualization, and simple machine learning tools from **scikit-learn** to better understand which factors are related to NBA scoring performance.

NBA teams use data analysis to evaluate players, compare performance, and make decisions about player development. In this project, I focus on player-season statistics such as points, rebounds, assists, true shooting percentage, usage percentage, age, height, and weight.

Data Source and Citation

The dataset used in this project is the **NBA Players Dataset** from Kaggle. It contains player-season data for NBA players, including demographic variables and performance statistics.

Source: **Kaggle - NBA Players Dataset by Justinas Cirtautas**

URL: <https://www.kaggle.com/datasets/justinas/nba-players-data>

Research Questions

1. What is the distribution of NBA players' points per game?
2. Is there a relationship between usage percentage and points per game?
3. Does shooting efficiency, measured by true shooting percentage, influence scoring?
4. How does scoring differ across age groups?
5. How has NBA scoring changed over time?
6. Which player attributes are most strongly correlated with scoring?
7. Can player performance statistics be used to predict points per game?

1. Import Libraries

The first step is to import the Python libraries needed for the project. I use **pandas** for data manipulation, **numpy** for numerical work, **matplotlib** for graphs, and **scikit-learn** for the prediction model.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Make plots display clearly
plt.rcParams["figure.figsize"] = (8, 5)
plt.rcParams["axes.grid"] = True
```

2. Load the Data

The dataset is stored in a CSV file called `all_seasons.csv`. This file should be in the same folder as the Jupyter notebook so the code can load it correctly.

```
In [2]: nba = pd.read_csv("all_seasons.csv")
nba.head()
```

```
Out[2]:
```

	Unnamed: 0	player_name	team_abbreviation	age	player_height	player_weight	co
0	0	Randy Livingston	HOU	22.0	193.04	94.800728	Loui
1	1	Gaylon Nickerson	WAS	28.0	190.50	86.182480	Northwe Okla
2	2	George Lynch	VAN	26.0	203.20	103.418976	I Cal
3	3	George McCloud	LAL	30.0	203.20	102.058200	Florida
4	4	George Zidek	DEN	23.0	213.36	119.748288	

5 rows × 22 columns



3. Quick Overview of the Dataset

Before cleaning or analyzing the data, I first check the size of the dataset, the column names, and the data types. This helps me understand what variables are available and whether some columns need to be converted or cleaned.

```
In [3]: print("Rows and columns:", nba.shape)
print()
```

```
print("Column names:")
print(nba.columns.tolist())
```

Rows and columns: (12844, 22)

Column names:

```
['Unnamed: 0', 'player_name', 'team_abbreviation', 'age', 'player_height', 'player_weight', 'college', 'country', 'draft_year', 'draft_round', 'draft_number', 'gp', 'pts', 'reb', 'ast', 'net_rating', 'oreb_pct', 'dreb_pct', 'usg_pct', 'ts_pct', 'ast_pct', 'season']
```

In [4]: `nba.info()`

```
<class 'pandas.DataFrame'>
RangeIndex: 12844 entries, 0 to 12843
Data columns (total 22 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            12844 non-null  int64
1   player_name           12844 non-null  str
2   team_abbreviation     12844 non-null  str
3   age                   12844 non-null  float64
4   player_height         12844 non-null  float64
5   player_weight         12844 non-null  float64
6   college               10990 non-null  str
7   country               12844 non-null  str
8   draft_year            12844 non-null  str
9   draft_round           12844 non-null  str
10  draft_number          12844 non-null  str
11  gp                    12844 non-null  int64
12  pts                   12844 non-null  float64
13  reb                   12844 non-null  float64
14  ast                   12844 non-null  float64
15  net_rating            12844 non-null  float64
16  oreb_pct              12844 non-null  float64
17  dreb_pct              12844 non-null  float64
18  usg_pct               12844 non-null  float64
19  ts_pct                12844 non-null  float64
20  ast_pct               12844 non-null  float64
21  season                12844 non-null  str
dtypes: float64(12), int64(2), str(8)
memory usage: 2.2 MB
```

In [5]: `nba.describe()`

Out[5]:

	Unnamed: 0	age	player_height	player_weight	gp	pts
count	12844.000000	12844.000000	12844.000000	12844.000000	12844.000000	12844.000000
mean	6421.500000	27.045313	200.555097	100.263279	51.154158	8.212580
std	3707.887763	4.339211	9.111090	12.426628	25.084904	6.016570
min	0.000000	18.000000	160.020000	60.327736	1.000000	0.000000
25%	3210.750000	24.000000	193.040000	90.718400	31.000000	3.600000
50%	6421.500000	26.000000	200.660000	99.790240	57.000000	6.700000
75%	9632.250000	30.000000	208.280000	108.862080	73.000000	11.500000
max	12843.000000	44.000000	231.140000	163.293120	85.000000	36.100000

4. Data Wrangling, Cleaning, and Feature Creation

For the cleaning step, I remove the unnecessary index column if it exists, convert height and weight into more familiar units, create age groups, and extract the starting year of each NBA season. I also filter out players who played fewer than 10 games and remove invalid shooting or usage percentage values. This helps reduce noise from players who barely appeared during a season.

```
In [6]: nba_clean = nba.copy()

# Remove extra index columns if they exist
nba_clean = nba_clean.drop(columns=[col for col in nba_clean.columns if "Unnamed" in col])

# Create easier-to-interpret variables
nba_clean["height_inches"] = nba_clean["player_height"] / 2.54
nba_clean["weight_pounds"] = nba_clean["player_weight"] * 2.20462
nba_clean["season_start"] = nba_clean["season"].str[:4].astype(int)

# Create age groups
nba_clean["age_group"] = pd.cut(
    nba_clean["age"],
    bins=[0, 24, 29, 34, 100],
    labels=["Under 25", "25-29", "30-34", "35+"]
)

# Filter data to keep meaningful player-season records
nba_clean = nba_clean[
    (nba_clean["gp"] >= 10) &
    (nba_clean["ts_pct"] > 0) &
    (nba_clean["ts_pct"] <= 1) &
    (nba_clean["usg_pct"] > 0)
].copy()
```

```
print("Cleaned data shape:", nba_clean.shape)
nba_clean.head()
```

Cleaned data shape: (11655, 25)

Out[6]:

	player_name	team_abbreviation	age	player_height	player_weight	college	country
0	Randy Livingston	HOU	22.0	193.04	94.800728	Louisiana State	US
2	George Lynch	VAN	26.0	203.20	103.418976	North Carolina	US
3	George McCloud	LAL	30.0	203.20	102.058200	Florida State	US
4	George Zidek	DEN	23.0	213.36	119.748288	UCLA	US
5	Gerald Wilkins	ORL	33.0	198.12	102.058200	Tennessee-Chattanooga	US

5 rows × 25 columns



Missing Values

This table checks the number of missing values in each column after the cleaning step. This is useful because missing values can affect plots, summary statistics, and machine learning models.

```
In [7]: nba_clean.isna().sum().sort_values(ascending=False)
```

```
Out[7]: college      1728
player_name         0
team_abbreviation   0
player_height       0
age                 0
player_weight       0
country             0
draft_year          0
draft_round         0
draft_number        0
gp                  0
pts                 0
reb                 0
ast                 0
net_rating          0
oreb_pct            0
dreb_pct            0
usg_pct             0
ts_pct              0
ast_pct             0
season              0
height_inches       0
weight_pounds       0
season_start        0
age_group           0
dtype: int64
```

Summary Statistics After Cleaning

The following summary gives an overview of the main numeric variables used in this project.

```
In [8]: important_vars = [
        "age", "height_inches", "weight_pounds", "gp", "pts", "reb", "ast",
        "net_rating", "usg_pct", "ts_pct", "ast_pct"
        ]

nba_clean[important_vars].describe()
```

Out[8]:

	age	height_inches	weight_pounds	gp	pts	
count	11655.000000	11655.000000	11655.000000	11655.000000	11655.000000	11655.000000
mean	27.110940	78.991577	221.398975	55.893436	8.757666	3.7681
std	4.350402	3.574064	27.317727	21.217005	5.963037	2.4553
min	18.000000	63.000000	132.999733	10.000000	0.100000	0.0000
25%	24.000000	76.000000	199.999599	40.000000	4.200000	2.0000
50%	26.000000	79.000000	219.999559	61.000000	7.300000	3.2000
75%	30.000000	82.000000	239.999519	74.000000	12.000000	4.9000
max	43.000000	91.000000	342.999312	85.000000	36.100000	16.1000

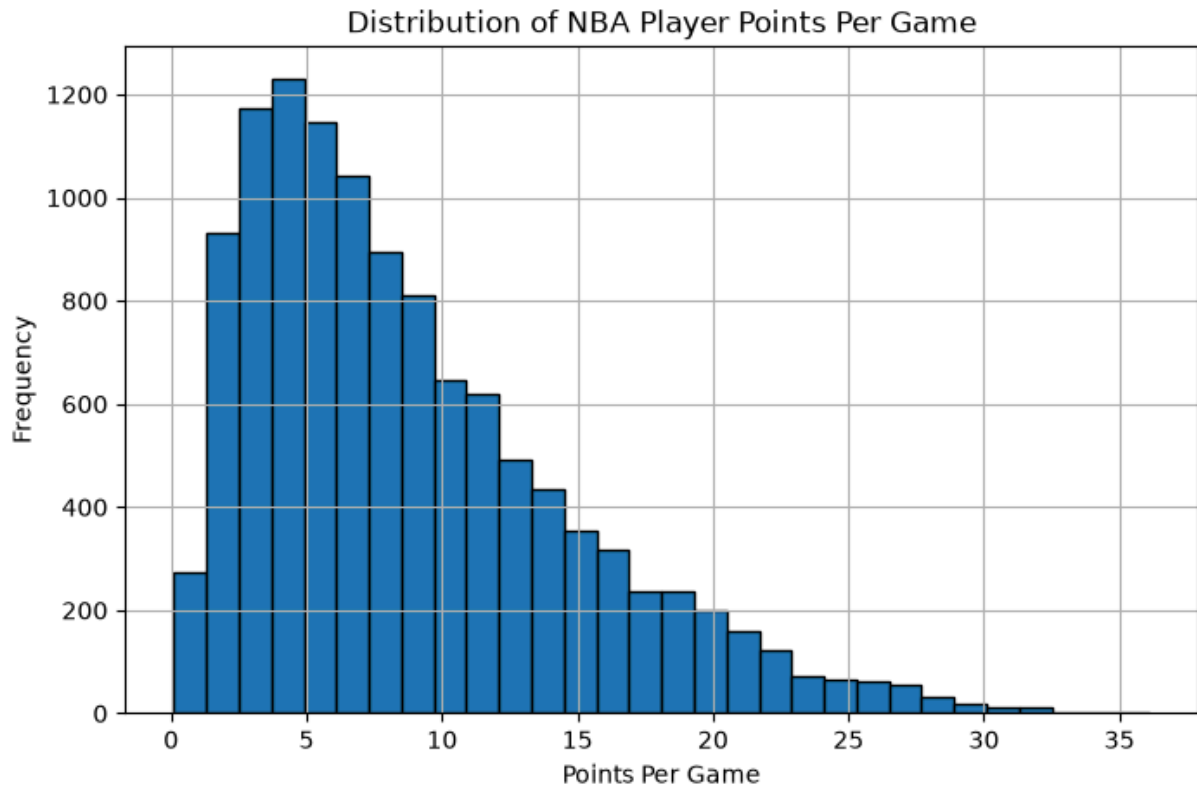
Exploratory Data Analysis

The following sections answer the seven research questions using visualizations, summary tables, correlation analysis, and predictive modeling.

Q1. What is the distribution of NBA players' points per game?

This question examines how scoring is distributed among NBA players. A histogram is useful here because it shows whether most players score similar amounts or whether only a small group of players score at a high level.

```
In [12]: plt.figure(figsize=(8, 5))
plt.hist(nba_clean["pts"], bins=30, edgecolor="black")
plt.title("Distribution of NBA Player Points Per Game")
plt.xlabel("Points Per Game")
plt.ylabel("Frequency")
plt.show()
```



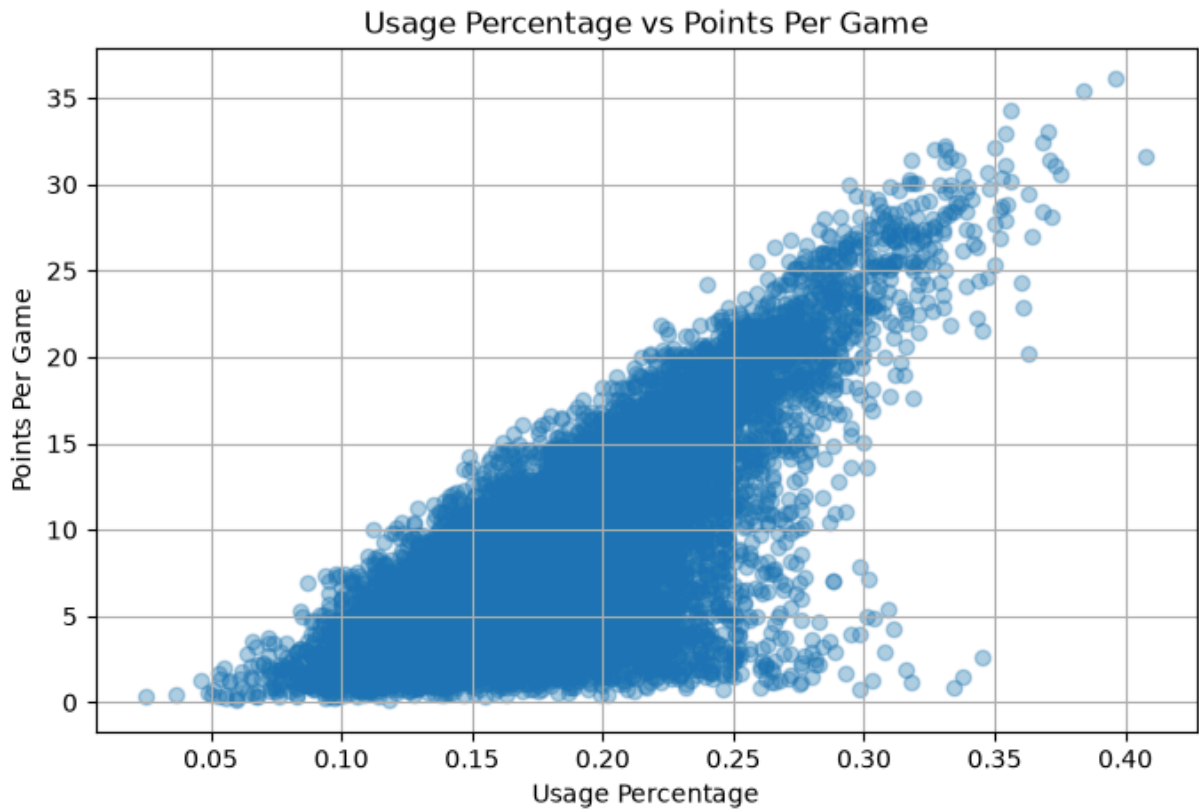
Q1 Interpretation

The histogram shows that points per game is right-skewed. Most NBA players score a smaller number of points per game, while only a smaller group of players average high scoring totals. This makes sense because NBA teams usually have only a few main scoring options, while many players have smaller offensive roles.

Q2. Is there a relationship between usage percentage and points per game?

Usage percentage estimates how involved a player is in the team's offense. This question investigates whether players with higher offensive involvement tend to score more points.

```
In [13]: plt.figure(figsize=(8, 5))
plt.scatter(nba_clean["usg_pct"], nba_clean["pts"], alpha=0.35)
plt.title("Usage Percentage vs Points Per Game")
plt.xlabel("Usage Percentage")
plt.ylabel("Points Per Game")
plt.show()
```

```
In [14]: usage_corr = nba_clean["usg_pct"].corr(nba_clean["pts"])
print("Correlation between usage percentage and points per game:", round(usage_corr
```

Correlation between usage percentage and points per game: 0.735

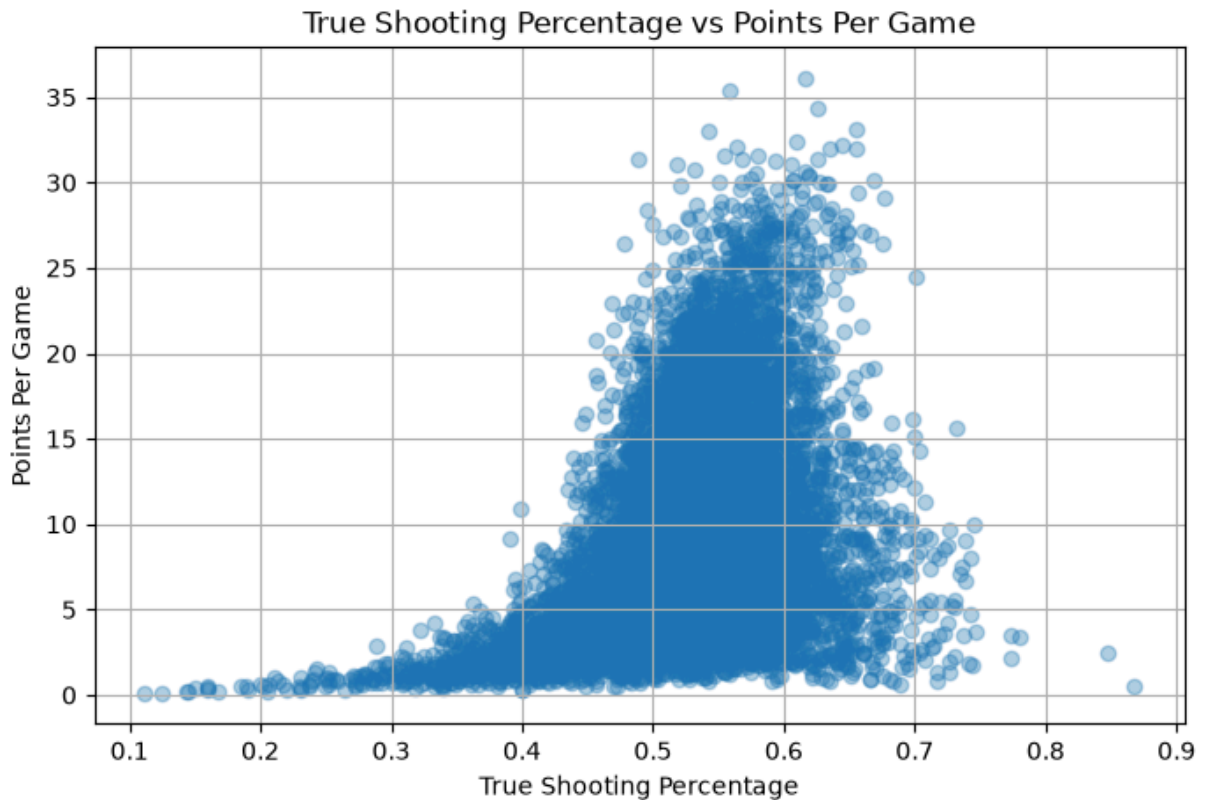
Q2 Interpretation

The scatter plot and correlation suggest a strong positive relationship between usage percentage and points per game. Players who are more involved in offensive possessions generally score more. This is expected because high-usage players are usually the players taking more shots or creating more offense for their teams.

Q3. Does shooting efficiency, measured by true shooting percentage, influence scoring?

True shooting percentage measures scoring efficiency by considering field goals, three-pointers, and free throws. This question looks at whether efficient scorers also tend to score more points per game.

```
In [15]: plt.figure(figsize=(8, 5))
plt.scatter(nba_clean["ts_pct"], nba_clean["pts"], alpha=0.35)
plt.title("True Shooting Percentage vs Points Per Game")
plt.xlabel("True Shooting Percentage")
plt.ylabel("Points Per Game")
plt.show()
```



```
In [16]: ts_corr = nba_clean["ts_pct"].corr(nba_clean["pts"])
print("Correlation between true shooting percentage and points per game:", round(ts
```

Correlation between true shooting percentage and points per game: 0.398

Q3 Interpretation

The relationship between true shooting percentage and points per game is positive, but it is not as strong as the relationship between usage percentage and scoring. This suggests that efficiency matters, but a player also needs enough offensive opportunities to become a high scorer.

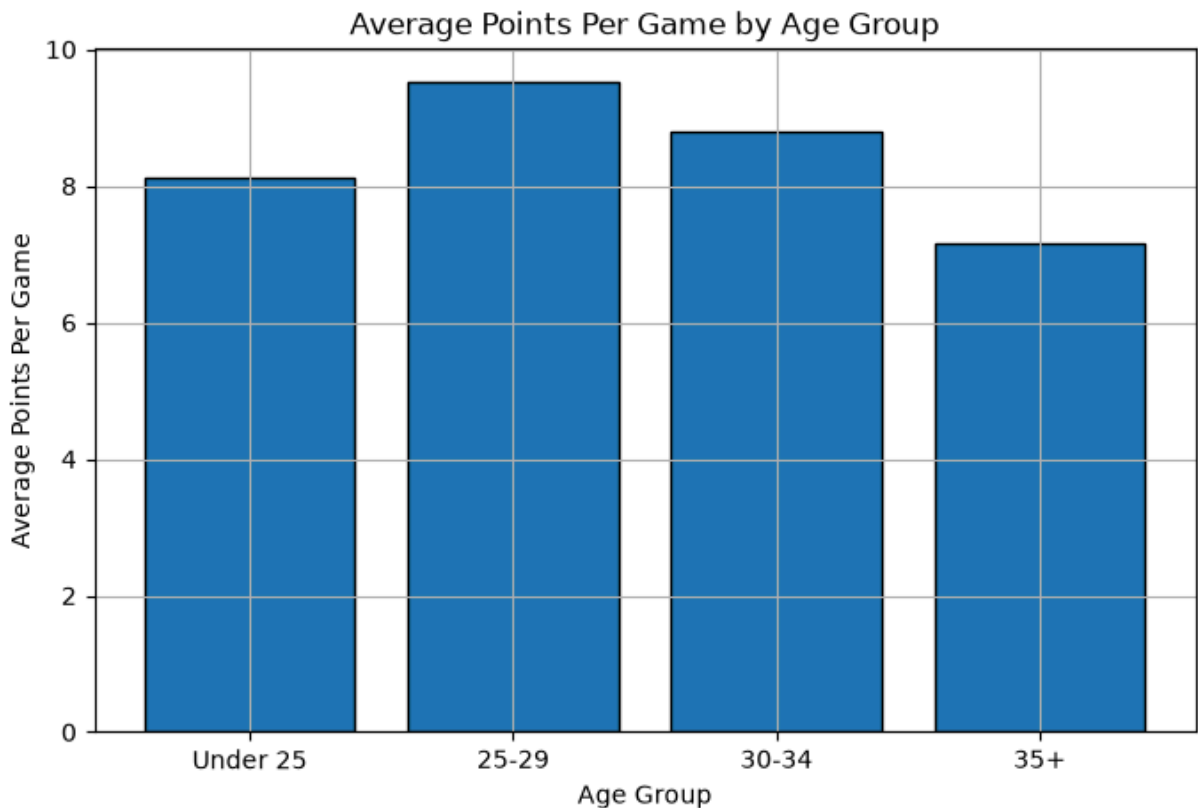
Q4. How does scoring differ across age groups?

This question compares scoring averages across four age groups. This helps show whether players in certain parts of their careers tend to produce more points.

```
In [17]: age_points = nba_clean.groupby("age_group", observed=True)["pts"].mean()

plt.figure(figsize=(8, 5))
plt.bar(age_points.index.astype(str), age_points.values, edgecolor="black")
plt.title("Average Points Per Game by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Average Points Per Game")
plt.show()

age_points
```



```
Out[17]: age_group
Under 25    8.129678
25-29      9.526748
30-34      8.796764
35+        7.166030
Name: pts, dtype: float64
```

Q4 Interpretation

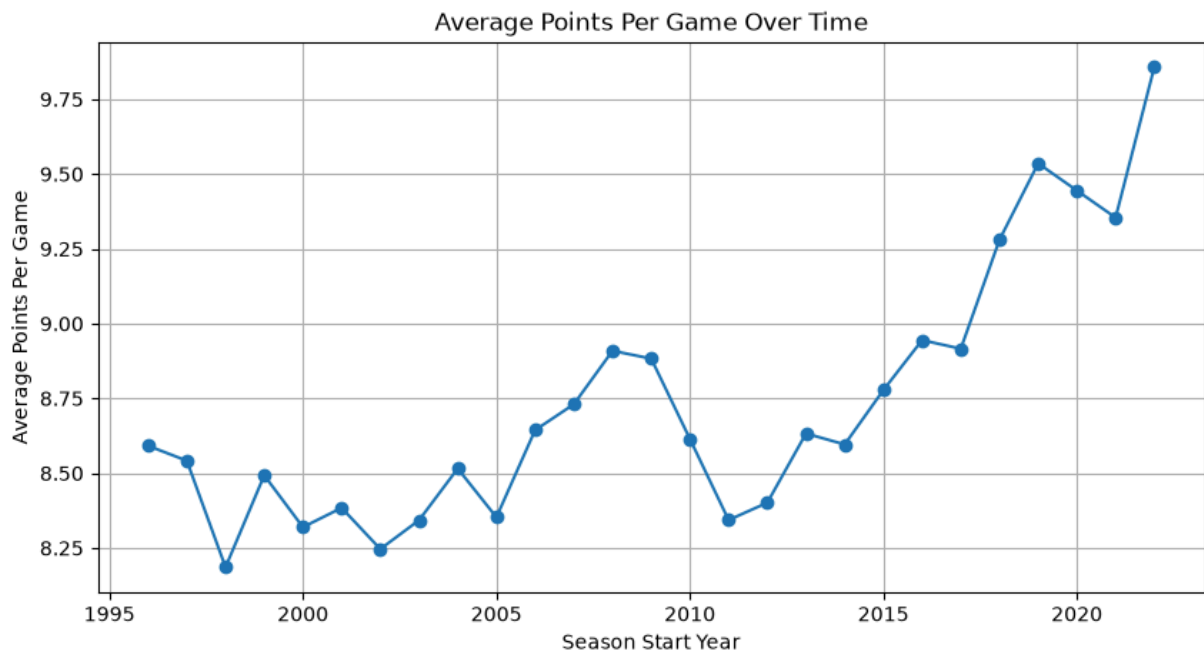
The age group comparison shows how scoring changes at different stages of a player's career. Players in their prime years often have higher scoring averages because they combine experience, athletic ability, and larger team roles. Younger players may still be developing, while older players may have reduced minutes or smaller roles.

Q5. How has NBA scoring changed over time?

This question studies whether average points per game has changed across seasons. Since the NBA has changed in style over time, especially with increased three-point shooting and faster offensive pace, this is an important trend to examine.

```
In [18]: scoring_by_season = nba_clean.groupby("season_start")["pts"].mean()

plt.figure(figsize=(10, 5))
plt.plot(scoring_by_season.index, scoring_by_season.values, marker="o")
plt.title("Average Points Per Game Over Time")
plt.xlabel("Season Start Year")
plt.ylabel("Average Points Per Game")
plt.show()
```



```
In [19]: print("Lowest average scoring season:")
print(scoring_by_season.sort_values().head(1))

print()
print("Highest average scoring season:")
print(scoring_by_season.sort_values(ascending=False).head(1))
```

```
Lowest average scoring season:
season_start
1998      8.185714
Name: pts, dtype: float64
```

```
Highest average scoring season:
season_start
2022      9.857234
Name: pts, dtype: float64
```

Q5 Interpretation

The line plot shows how average scoring has changed over time. If the later seasons are higher than the earlier seasons, that supports the idea that scoring has increased in the modern NBA. This may be connected to changes in offensive strategy, pace of play, and the increased importance of three-point shooting.

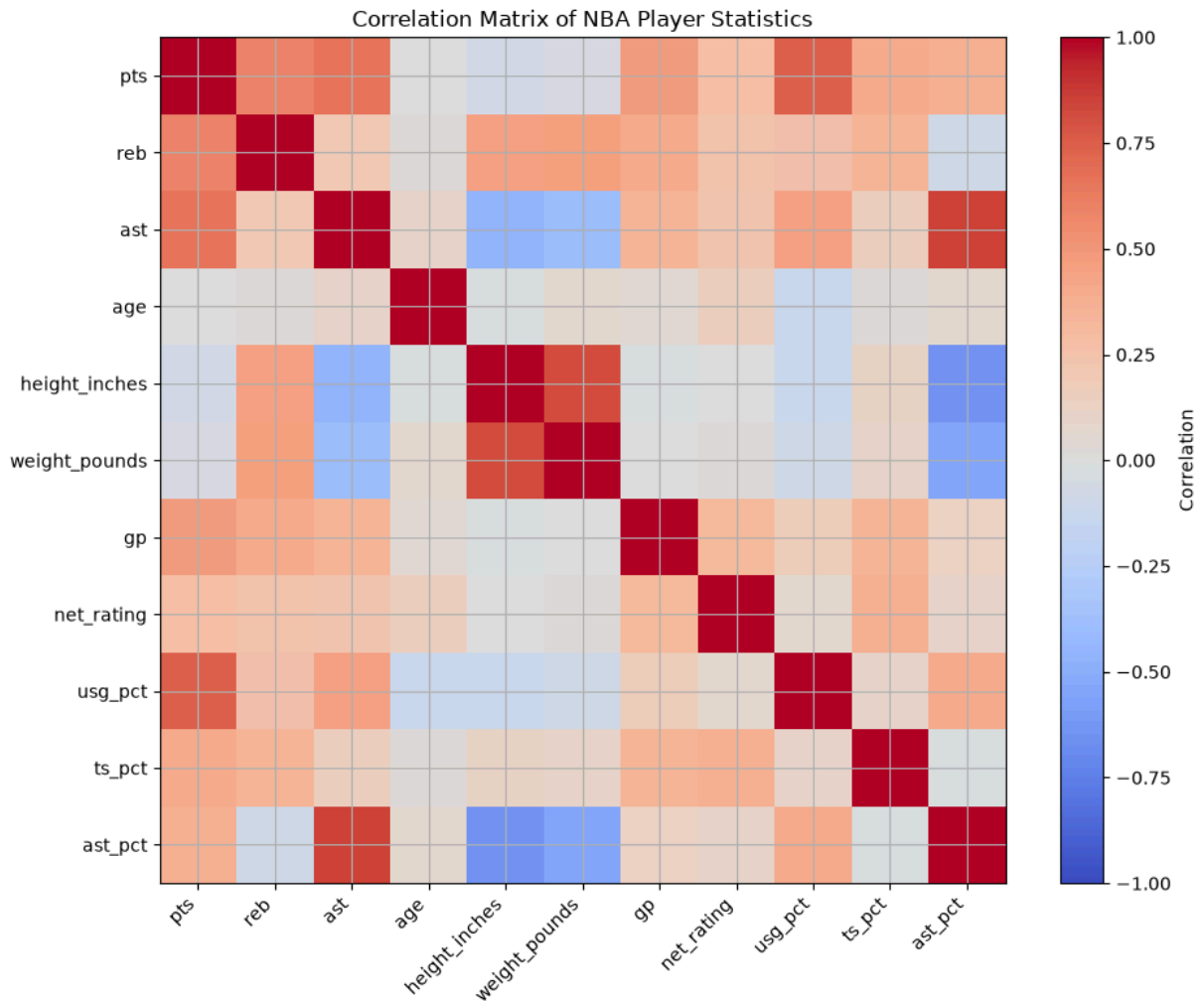
Q6. Which player attributes are most strongly correlated with scoring?

Correlation analysis helps identify which variables move most closely with points per game. This does not prove causation, but it helps show which variables are most strongly related to scoring.

```
In [20]: corr_vars = [
    "pts", "reb", "ast", "age", "height_inches", "weight_pounds",
    "gp", "net_rating", "usg_pct", "ts_pct", "ast_pct"
]

corr_matrix = nba_clean[corr_vars].corr()

plt.figure(figsize=(10, 8))
plt.imshow(corr_matrix, cmap="coolwarm", vmin=-1, vmax=1)
plt.colorbar(label="Correlation")
plt.xticks(range(len(corr_vars)), corr_vars, rotation=45, ha="right")
plt.yticks(range(len(corr_vars)), corr_vars)
plt.title("Correlation Matrix of NBA Player Statistics")
plt.tight_layout()
plt.show()
```



```
In [21]: pts_correlations = corr_matrix["pts"].sort_values(ascending=False)
pts_correlations
```

```
Out[21]: pts          1.000000
usg_pct      0.734978
ast          0.648450
reb          0.592122
gp           0.483227
ts_pct       0.397527
ast_pct      0.359860
net_rating   0.285131
age          -0.003124
weight_pounds -0.038549
height_inches -0.066881
Name: pts, dtype: float64
```

Q6 Interpretation

The correlation results show which variables are most related to points per game. Usage percentage is expected to have one of the strongest positive relationships with scoring because players who use more possessions usually score more. Assists and rebounds may also be positively related because higher-quality players often contribute in multiple

statistical categories. Physical variables such as height and weight may matter more for rebounding than scoring.

Q7. Can player performance statistics be used to predict points per game?

For the final question, I build prediction models using player statistics as input variables. This upgrades the project from only exploratory analysis to a simple machine learning task. I use both a linear regression model and a random forest model, then compare their performance using RMSE and R-squared.

- **RMSE** measures the typical prediction error in points per game. Lower RMSE is better.
- **R-squared** measures how much variation in points per game is explained by the model. Higher R-squared is better.

```
In [22]: features = [
    "usg_pct", "ts_pct", "gp", "age", "height_inches",
    "weight_pounds", "reb", "ast", "net_rating", "ast_pct"
]

X = nba_clean[features]
y = nba_clean["pts"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training set size:", X_train.shape)
print("Testing set size:", X_test.shape)
```

Training set size: (9324, 10)

Testing set size: (2331, 10)

```
In [23]: # Linear Regression Model
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

linear_predictions = linear_model.predict(X_test)
linear_rmse = np.sqrt(mean_squared_error(y_test, linear_predictions))
linear_r2 = r2_score(y_test, linear_predictions)

print("Linear Regression RMSE:", round(linear_rmse, 3))
print("Linear Regression R-squared:", round(linear_r2, 3))
```

Linear Regression RMSE: 1.959

Linear Regression R-squared: 0.895

```
In [24]: linear_coefficients = pd.DataFrame({
    "Variable": features,
    "Coefficient": linear_model.coef_
}).sort_values(by="Coefficient", ascending=False)
```

```
linear_coefficients
```

Out[24]:

	Variable	Coefficient
0	usg_pct	62.916305
1	ts_pct	11.519557
7	ast	2.137128
6	reb	0.684024
3	age	0.039803
2	gp	0.019943
8	net_rating	0.004285
5	weight_pounds	-0.019850
4	height_inches	-0.120348
9	ast_pct	-32.471143

In [25]:

```
# Random Forest Model
rf_model = RandomForestRegressor(n_estimators=100, random_state=42, max_depth=8)
rf_model.fit(X_train, y_train)

rf_predictions = rf_model.predict(X_test)
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_predictions))
rf_r2 = r2_score(y_test, rf_predictions)

print("Random Forest RMSE:", round(rf_rmse, 3))
print("Random Forest R-squared:", round(rf_r2, 3))
```

Random Forest RMSE: 1.542

Random Forest R-squared: 0.935

In [26]:

```
model_comparison = pd.DataFrame({
    "Model": ["Linear Regression", "Random Forest"],
    "RMSE": [linear_rmse, rf_rmse],
    "R-squared": [linear_r2, rf_r2]
})

model_comparison
```

Out[26]:

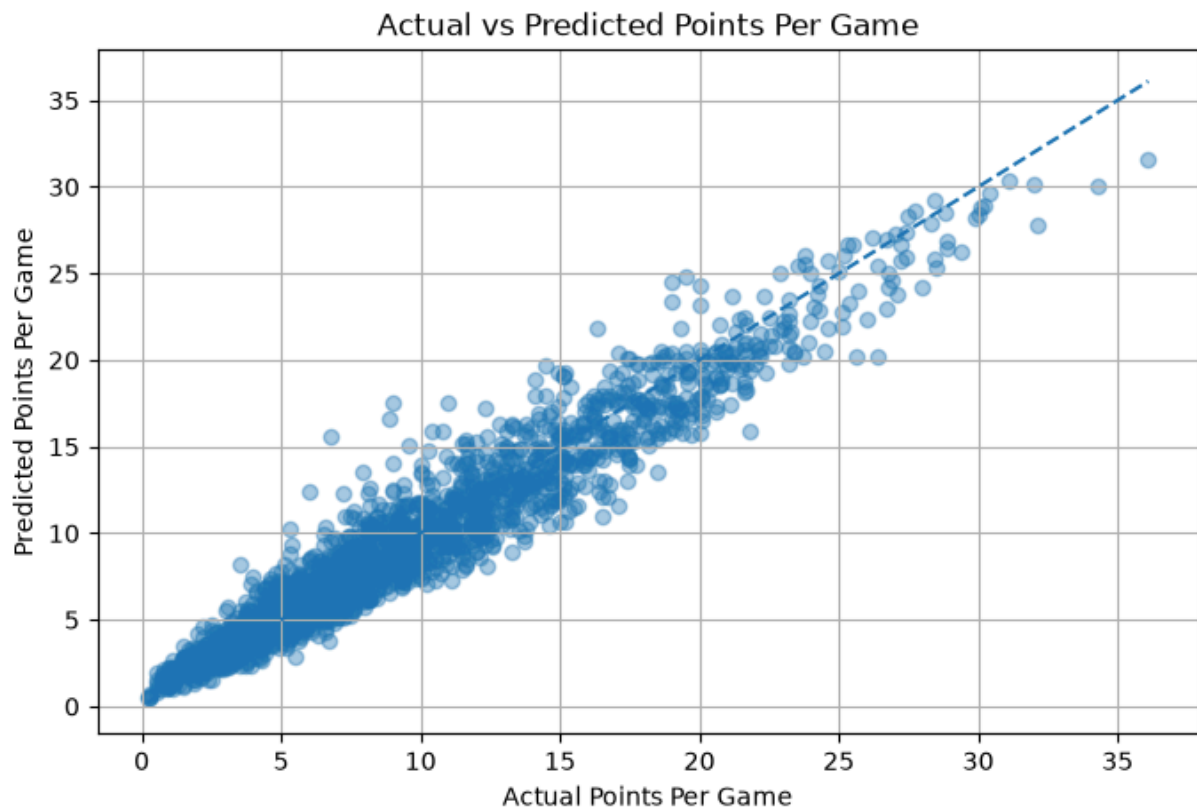
	Model	RMSE	R-squared
0	Linear Regression	1.958596	0.894786
1	Random Forest	1.541735	0.934807

In [27]:

```
plt.figure(figsize=(8, 5))
plt.scatter(y_test, rf_predictions, alpha=0.4)
plt.title("Actual vs Predicted Points Per Game")
plt.xlabel("Actual Points Per Game")
```



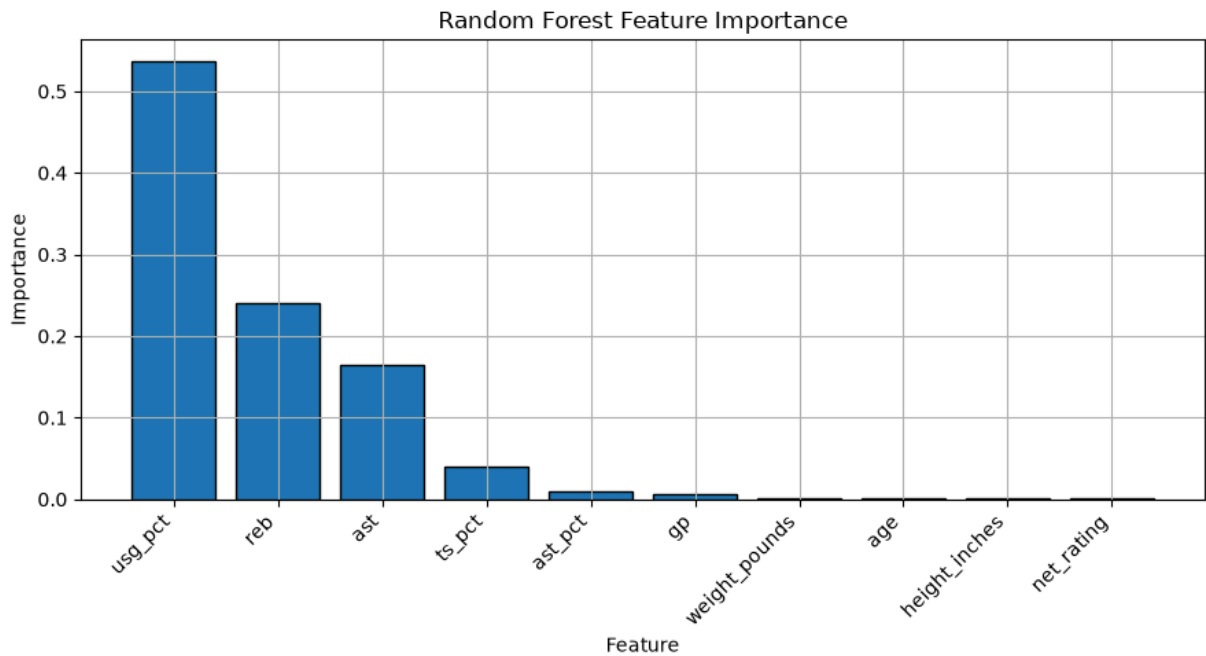
```
plt.ylabel("Predicted Points Per Game")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], linestyle="--")
plt.show()
```



```
In [28]: feature_importance = pd.DataFrame({
    "Feature": features,
    "Importance": rf_model.feature_importances_
}).sort_values(by="Importance", ascending=False)

plt.figure(figsize=(9, 5))
plt.bar(feature_importance["Feature"], feature_importance["Importance"], edgecolor=
plt.title("Random Forest Feature Importance")
plt.xlabel("Feature")
plt.ylabel("Importance")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

feature_importance
```



Out[28]:

	Feature	Importance
0	usg_pct	0.536501
6	reb	0.240288
7	ast	0.164473
1	ts_pct	0.039424
9	ast_pct	0.009696
2	gp	0.005850
5	weight_pounds	0.001430
3	age	0.000848
4	height_inches	0.000757
8	net_rating	0.000734

Q7 Interpretation

The machine learning models show that player statistics can be used to predict points per game reasonably well. If the R-squared value is high, then the model explains a large amount of the variation in scoring. The feature importance chart shows which variables contributed the most to the prediction. Usage percentage is expected to be one of the most important predictors because scoring is closely connected to how often a player is involved in the offense.

Final Conclusion

This project used Python to analyze factors influencing NBA player scoring performance. The analysis showed that points per game is not evenly distributed across players; most players score modestly, while a smaller number of players are high-volume scorers. Usage percentage had one of the strongest relationships with points per game, which makes sense because players who are more involved in their team's offense usually have more chances to score.

True shooting percentage also had a positive relationship with scoring, but efficiency alone was not enough to explain scoring output. Age group comparisons suggested that players in their prime years tend to have higher scoring averages. The time trend analysis showed how scoring has changed across NBA seasons, giving context to how the league has evolved.

The predictive modeling section showed that player statistics can be used to estimate points per game. The random forest model and linear regression model both provided useful results, and the feature importance analysis helped identify the variables most connected to scoring.

Overall, this project demonstrates how pandas, matplotlib, and machine learning tools in Python can be used to clean data, explore patterns, create visualizations, and answer real-world sports analytics questions.

Limitations

There are a few limitations in this project. First, the dataset is based on season-level statistics, so it does not include game-by-game context. Second, the dataset does not include every factor that affects scoring, such as injuries, team strategy, opponent defense, or player position. Finally, correlation and prediction models can show relationships, but they do not prove direct causation.